System / Architecture Sketch

Components, data flow and the hybrid downscaling model behind PanchayatCast

A. High-Level Architecture

B. Component Summary

Component	Technology	Responsibility
Frontend	Next.js, Tailwind CSS, Leaflet, Recharts, next-intl	Officer dashboard, split map, panchayat detail, advisory, validation; mobile farmer page
Backend / API	FastAPI, Pydantic, Uvicorn	REST endpoints, GeoJSON output, advisory generation, share simulator, JWT auth
Database	PostgreSQL 15 + PostGIS	Panchayat polygons, daily forecasts per GP, advisories, validation scores, users
Storage	Local volume / S3-compatible bucket (COG files)	1 km gridded outputs, trained models, cached raw inputs
Downscaling model	xarray, rasterio, scikit-learn, XGBoost, SciPy	Hybrid statistical + ML downscaling (details in section D)
Scheduler	APScheduler (in backend container)	Runs daily pipeline; retries on failure; Demo Mode cache
External services	IMD feeds, NCEP GFS, ERA5-Land, CHIRPS, SRTM, OSM tiles	Forecast input, training data, static features, base maps
Auth	JWT (officer / admin roles)	Officers log in; farmers use a public read-only panchayat link - no login barrier
Deployment	Docker Compose; Vercel (frontend) + Render (API + DB)	One-command local run for judging; cloud URL as backup

C. Data Flow (Daily Run)

1. **Ingest (06:00):** adapters pull the latest IMD block-level 5-day forecast (or GFS fallback), validate the schema and cache the raw files.
2. **Preprocess:** forecasts are aligned to a common 1 km grid; static layers (elevation, slope, land use, NDVI, distance to water, monthly climatology) are pre-computed once and reused.
3. **Downscale:** the hybrid engine produces 1 km fields for Tmax, Tmin, rainfall (amount + probability), RH and wind for Days 1-5, then rescales them so the block average matches IMD's official value.
4. **Aggregate:** zonal statistics per panchayat polygon (mean / min / max, rain probability) plus a High / Medium / Low confidence label from model spread.
5. **Advise:** the rule engine combines panchayat forecasts with crop and growth stage to create ranked advisories in Telugu, Hindi and English.
6. **Store & serve:** results are written to PostGIS and the file store; FastAPI serves them to the dashboard and the farmer page; officers push advisories through the notification service.
7. **Validate:** when AWS observations arrive, the system scores yesterday's forecast vs the block baseline and updates the validation view.

D. Downscaling Model Design (Statistical + ML Hybrid)

Variable	Method	Key predictors
Tmax / Tmin	Lapse-rate correction (-6.5 °C/km vs block mean elevation) + XGBoost on residuals	Elevation diff., slope, aspect, LULC, NDVI, distance to water, climatology
Rainfall	Two-stage: rain/no-rain classifier + amount regressor, then quantile mapping; output = amount + probability	Coarse rain, climatological rain ratio, elevation, terrain exposure
Relative humidity	Derived from downscaled temperature and dew point for physical consistency	Downscaled T, block dew point
Wind speed	Terrain exposure + surface-roughness scaling	Topographic position index, LULC roughness

Training strategy - perfect prognosis: models learn the mapping from coarse historical fields (IMD gridded / ERA5) to fine fields (ERA5-Land, CHIRPS) over 15-20 years, then are applied to live forecasts. This avoids needing a long archive of past forecasts. AWS stations are held out purely for independent validation.